

SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

CO1 ASSESSMENT TOOL 1– Case Study Analysis

Course Code:	CSA1001	Course Title:	Software Engineering
CO Assessed:	CO1	Assessment:	Assessment Tool 1 – Case Study Analysis
Weightage:	20%	Total Marks:	25
CO1 Statement:	Apply advanced methodologies and agile project management techniques to manage software projects effectively		
Student Name:	Jai Ganesh Rao N	Reg. No.:	192524073
Date:	01-10-2026	Duration:	60 minutes

Code of Conduct

I Jai Ganesh Rao N/ 192524073 certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: Jai Ganesh Rao N

Annexure A – Case Study Analysis

Instructions to Students:

Each student will be allotted **one problem statement**. Analyze the assigned problem systematically by following the steps given below. Your report should demonstrate your understanding of software engineering concepts and your ability to design an appropriate software solution.

Based on your allotted problem statement, prepare a report by answering the following questions:

Dark web threat actor de-anonymization

1. Understand the Problem Statement

Problem Explanation

The dark web provides anonymity to users through technologies such as Tor hidden services. This makes it difficult for cybersecurity investigators to identify and connect threat actors involved in illegal activities.

The proposed system aims to collect information about threat actors from multiple sources such as dark-web marketplaces, forums, and other publicly available intelligence sources. The collected information can include usernames or handles, PGP keys, cryptocurrency wallet identifiers, infrastructure indicators, and relationships between different online identities.

The system will analyze these indicators and create connections between different identities. It will also use AI-based techniques such as **stylographic analysis and behavioural profiling** to identify possible connections between personas that may belong to the same threat actor. The system is intended to support authorized cybersecurity investigations by providing an analytical platform rather than relying on a single indicator.

Objectives

The main objectives are:

1. Collect threat-actor intelligence from multiple reliable sources.
2. Maintain information about handles, PGP keys, wallets and other identifiers.
3. Identify relationships between different online personas.
4. Detect infrastructure-related indicators associated with hidden services.
5. Apply AI-based analysis to identify possible persona relationships.
6. Assign an attribution confidence level to identified relationships.
7. Provide a searchable relationship graph and analytical dashboard.
8. Allow investigators to analyze information across a selected timeline.
9. Support autonomous and continuous intelligence collection.
10. Export investigation results in **CSV, JSON and report formats**.

Scope

The scope includes:

- Dark-web threat intelligence collection.
- Threat-actor profile management.
- Identifier and relationship management.
- Infrastructure-indicator analysis.
- Persona-link analysis.
- Timeline-based investigation.
- AI-assisted analysis.
- Relationship graph visualization.
- Confidence-based attribution.
- Dashboard and reporting.
- Data export.

The system should be designed for **authorized cybersecurity and law-enforcement investigations**.

2. Identify Key Stakeholders and Challenges

Stakeholder	Role	Major Challenges
Cybersecurity Investigators	Analyze threat actors and relationships	Large volume of data and difficult attribution
Law Enforcement Agencies	Investigate cybercrime	Need reliable evidence and proper authorization
Threat Intelligence Analysts	Collect and correlate intelligence	Different sources use different identifiers
Security Researchers	Study threat-actor behaviour	Rapidly changing personas and infrastructure
System Administrators	Maintain the platform	Availability, security and scalability
Data/AI Analysts	Develop analytical models	False positives and incomplete data
System Users/Investigators	Search and visualize intelligence	Complex data must be presented clearly
Organizations/Incident Response Teams	Use intelligence for security investigations	Need timely and trustworthy intelligence

Major Challenges

1. Anonymity

Threat actors deliberately use anonymous infrastructure and pseudonymous identities.

2. Multiple identities

One individual may use different usernames, wallets or PGP keys on different platforms.

3. Changing identities

Threat actors may abandon an account and create a new persona.

4. Data reliability

Information collected from online sources may be incomplete, outdated or intentionally misleading.

5. Large-scale data

Continuous collection can generate a very large amount of information.

6. False attribution

Two different people may use similar usernames or writing styles. Therefore, the system must not automatically treat a similarity as proof of identity.

7. Security and privacy

The intelligence database itself contains sensitive cybersecurity information and must be strongly protected.

3. Analyze the Current Approach

Existing Approach

Traditional threat-actor investigations generally involve analysts collecting information from different sources and manually correlating identifiers.

For example, an analyst may separately examine:

- Dark-web forum accounts
- Marketplace profiles
- PGP identities
- Cryptocurrency wallet identifiers
- Publicly observable infrastructure
- Previously identified threat-actor information

The investigator then attempts to determine whether multiple identities belong to the same actor.

Strengths

1. Human investigators can understand context.
2. Analysts can verify information from multiple sources.
3. Existing cybersecurity tools can provide individual pieces of intelligence.
4. Experienced investigators can recognize behavioural patterns.
5. Manual verification can reduce automated false attribution.

Limitations

However, a completely manual approach has several limitations:

- Time-consuming investigation.
- Difficult to process large quantities of data.
- Information is distributed across multiple sources.
- Relationships between identifiers can be difficult to visualize.
- Threat actors can frequently change their identities.
- Historical information may be difficult to correlate.
- Manual analysis can introduce inconsistency.
- Continuous monitoring is difficult.

Identified Gap

The major gap is the absence of a **single integrated platform** that can collect, contextualize, correlate and visualize threat-actor intelligence over time.

The proposed system addresses this gap by combining collection, database storage, relationship analysis, AI-assisted persona analysis and visualization into one platform.

4. Propose a Suitable Solution Using Software Engineering Principles

An AI-assisted **Dark Web Threat Actor Intelligence & Attribution Platform** designed to collect, process, analyze, and visualize intelligence on threat actors while ensuring security, scalability, and usability.

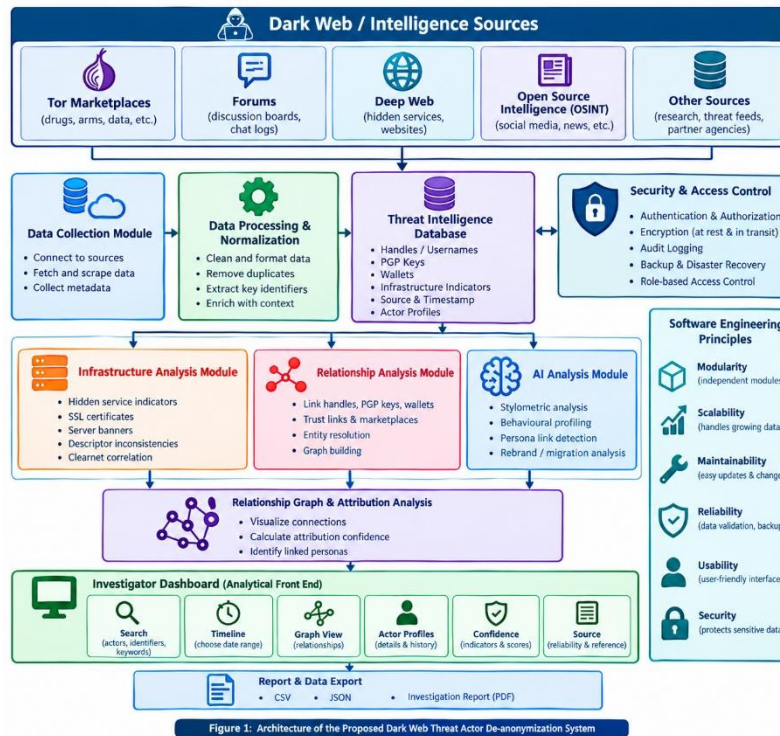


Figure 1: Architecture of the Proposed Dark Web Threat Actor De-anonymization System

Major Functional Modules

1. **Data Collection** – Gather handles, PGP keys, wallets, infra indicators, timestamps, and sources from authorized feeds.
2. **Data Processing** – Normalize formats, remove duplicates, extract entities, and structure intelligence.
3. **Threat Actor Profiles** – Maintain persona IDs, identifiers, infra links, activities, dates, confidence levels, and references.
4. **Relationship Analysis** – Map connections between actors, handles, wallets, and infrastructure.
5. **AI Analysis** – Detect writing style, behavioral patterns, terminology, posting habits, and timing similarities.
6. **Infrastructure Intelligence** – Record metadata, certificates, domains, banners, and historical infra observations.
7. **Timeline Analysis** – View actors and relationships across selected timeframes.
8. **Graph Visualization** – Interactive graphs to simplify complex relationships.

9. **Dashboard** – Unified interface with search, profiles, graphs, timelines, confidence indicators, infra data, and export options.
10. **Reporting/Export** – Generate CSV, JSON, and investigation reports.

Functional Requirements

- Collect and store intelligence with timestamps and sources.
- Maintain threat actor profiles and relationships.
- Enable multi-parameter search.
- Provide graph and timeline analysis.
- Perform AI-assisted persona attribution with confidence scoring.
- Export results in required formats.
- Continuously update intelligence feeds.

Non-Functional Requirements

- **Security** – Authentication, authorization, encryption, audit logs.
- **Scalability** – Handle growing data volume and actors.
- **Reliability** – Backups, error handling, high availability.
- **Performance** – Fast search and graph queries.
- **Maintainability** – Modular design for independent updates.
- **Usability** – Clear dashboard and visualization tools.
- **Auditability** – Log all critical actions.
- **Accuracy** – Distinguish confirmed, correlated, AI leads, and unverified intelligence.

Incorporation of Software Engineering Principles

- **Modularity** – Independent modules (collection, processing, AI, visualization, reporting) for easier updates and testing.
- **Scalability** – Flexible database and modular backend to support increasing intelligence volume.
- **Maintainability** – Clear separation of concerns; each component can be updated without affecting others.
- **Reliability** – Data validation, backups, logging, and error handling ensure system stability.
- **Usability** – Graphs and dashboards simplify complex relationships for investigators.
- **Security** – Strong authentication, encryption, secure storage, and audit trails protect sensitive intelligence.

5. Justify the Chosen Methodology/Framework

Selected Methodology: Agile

Agile development is suitable for this project because the system contains several complex modules and requirements may change as the project develops.

The project can be divided into iterations.

Sprint 1 — Basic Platform

Develop:

- User authentication
- Database
- Basic dashboard
- Threat-actor profile

Sprint 2 — Intelligence Collection

Develop:

- Source integration
- Data ingestion
- Data normalization
- Source tracking

Sprint 3 — Relationship Analysis

Develop:

- Entity relationships
- Graph database/graph processing
- Relationship visualization

Sprint 4 — AI Analysis

Develop:

- Stylometric analysis
- Behavioural analysis
- Confidence calculation

Sprint 5 — Reporting

Develop:

- Timeline analysis
- CSV export
- JSON export
- Investigation reports

Sprint 6 — Testing and Security

Perform:

- Functional testing
- Integration testing
- Security testing
- Performance testing
- User acceptance testing

Why Agile is Suitable

Agile allows the development team to develop the system incrementally. Instead of developing the entire complex platform at once, individual modules can be developed and tested in separate iterations. It also allows investigators and cybersecurity experts to provide feedback during development. The assessment asks for justification of the selected SDLC/framework and how it supports successful development and maintenance.

6. Expected Outcomes and Limitations

Expected Outcomes

The proposed system is expected to:

1. Develop an end-to-end Dark Web Threat Intelligence System for collecting, storing, and analyzing threat actor information.
2. Provide a GUI/dashboard for searching, querying, and visualizing threat intelligence.
3. Maintain information such as actor profiles, handles, PGP keys, wallets, infrastructure indicators, persona linkages, attribution confidence, category, scan date, and sources.
4. Enable contextualization and correlation of threat actor data for better analysis.
5. Provide secure and efficient data storage, retrieval, and management.
6. Allow users to export results in CSV, JSON, and report formats.
7. Apply key software engineering principles such as security, scalability, maintainability, reliability, and usability.
8. Demonstrate proper requirements analysis, system design, SDLC selection, and evaluation of risks, limitations, and future enhancements.

Limitations

1. No Guaranteed Attribution

A relationship between two online identities does not necessarily prove that they belong to the same real-world person.

2. False Positives

Similar writing styles, usernames or behaviours can occur among unrelated individuals.

3. False Negatives

The system may fail to identify a relationship when an actor changes their behaviour or uses completely different identifiers.

4. Incomplete Data

Some sources may disappear, become inaccessible or contain incomplete information.

5. AI Limitations

AI-based analysis can produce incorrect associations and therefore requires human verification.

6. Data Quality

The quality of the final intelligence depends heavily on the reliability of the underlying sources.

7. Legal and Ethical Constraints

Collection and processing of intelligence must follow applicable laws, authorization requirements, privacy rules and organizational policies.

8. Security Risk

A centralized intelligence database itself could become a valuable target for attackers.

Future Enhancements

Possible future improvements include:

- Advanced graph-based machine learning.
- Improved entity-resolution algorithms.
- Real-time intelligence feeds.
- More sophisticated anomaly detection.
- Improved multilingual stylometric analysis.
- Automated source reliability scoring.
- Historical infrastructure tracking.
- Advanced visualization.
- Integration with existing threat-intelligence platforms.
- Improved human-in-the-loop verification.

Rubrics for Calculation

Criteria	Excellent (5 Marks)	Good (3 Marks)	Satisfactory (2 Marks)	Needs Improvement (1 Mark)
Problem Understanding (30%)	Clearly explains the problem, objectives, and scope with complete understanding.	Explains the problem with minor omissions.	Basic understanding with limited explanation.	Poor understanding; objectives and scope are unclear or missing.
Stakeholder & Challenge Analysis (25%)	Identifies all stakeholders, their roles, and challenges with clear analysis.	Identifies most stakeholders and challenges with adequate analysis.	Identifies only a few stakeholders or challenges.	Stakeholders and system analysis are incomplete or inaccurate.
Current System Analysis (20%)	Thoroughly analyzes the existing system, strengths, weaknesses, and identifies all gaps.	Good analysis with most strengths and weaknesses identified.	Limited analysis with partial identification of issues.	Proposed solution is unclear or inappropriate; requirements are largely missing.
Proposed Software Solution & Software Engineering Principles (15%)	Proposes an innovative, feasible solution applying appropriate software engineering principles (modularity, scalability, maintainability, security, etc.).	Suitable solution with good application of software engineering principles.	Basic solution with limited application of software engineering principles.	Methodology is inappropriate, poorly justified, or not discussed.
Methodology/ Framework Justification (10%)	Selects the most suitable SDLC/model/framework and provides strong technical justification.	Suitable methodology selected with adequate justification.	Methodology selected but justification is weak.	Outcomes, limitations, or future enhancements are missing; report lacks organization and clarity.

Criteria	Mark
Problem Understanding (30%)	/5
Stakeholder & Challenge Analysis (25%)	/5
Current System Analysis (20%)	/5
Proposed Software Solution & Software Engineering Principles (15%)	/5
Methodology/ Framework Justification (10%)	/5
TOTAL	/25